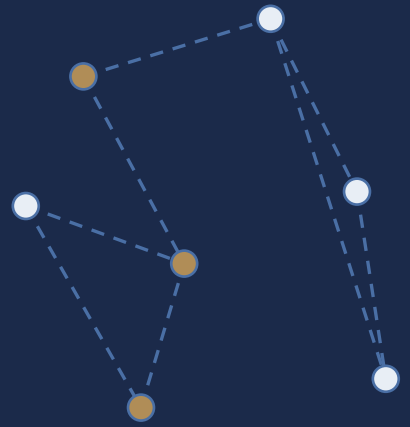
QiOpt4QNet



Quantum-Inspired Optimization for Entanglement Routing in Quantum Networks

From QUBO Solver Families to Chance-Constrained Stochastic Scheduling

Arnav Kapoor Manya Ahuja Joshua Tom Abhishek Rufus Raj Shlok Goenka
Mentors : Krishna Bhatia Shalini Devendrababu

github.com/Sguy1908/QiOpt4QNet-Simulator

Entanglement Routing as a Resource-Allocation Problem

- Quantum networks distribute entanglement between distant nodes
  the resource behind QKD, distributed quantum computing and blind quantum computing.
- Concurrent requests compete for scarce edge (Bell-pair) and node (memory-qubit) capacity.
- Entanglement generation and swapping are probabilistic
  stored pairs decay under T1/T2.
- End-to-end fidelity depends jointly on link quality, purification rounds, and swap order.
- Most prior work routes greedily or heuristically
  few cast the full request-allocation problem as a single energy-minimization (QUBO/Ising) objective.

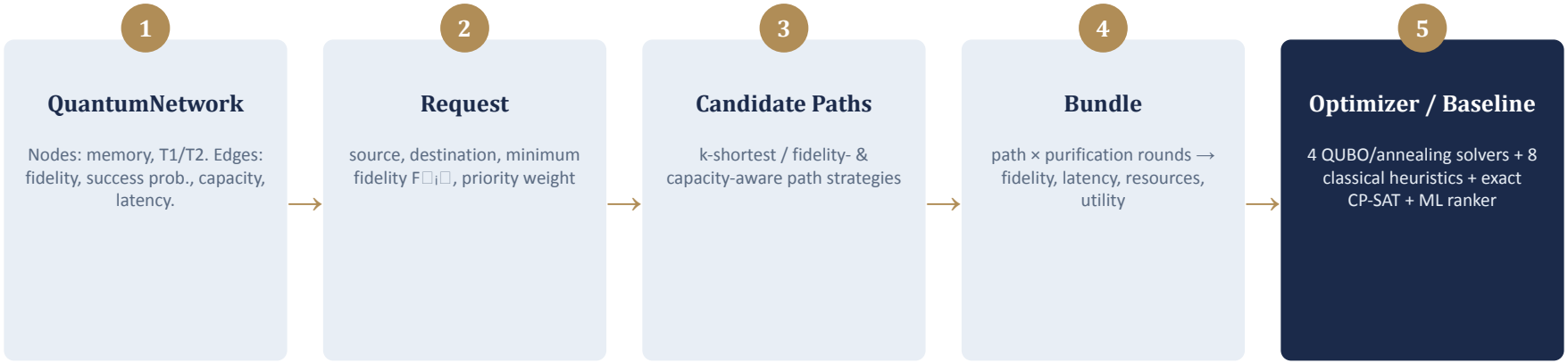
Repeater chain: lossy / noisy links, limited memory per node

A **Bundle** = one candidate path + a purification schedule for one request

Every Bundle carries

- End-to-end fidelity
- Latency & success probability
- Edge + memory resource demand
- A computed utility score

Core Data Model & Solver Pipeline



Uniform contract: every solver, quantum or classical, returns the same *{selected: [(request, bundle)…], total_utility}* shape, so any of them drop into the same comparison tooling interchangeably.

QuantumNode

memory capacity, T1/T2, reservations

QuantumLink

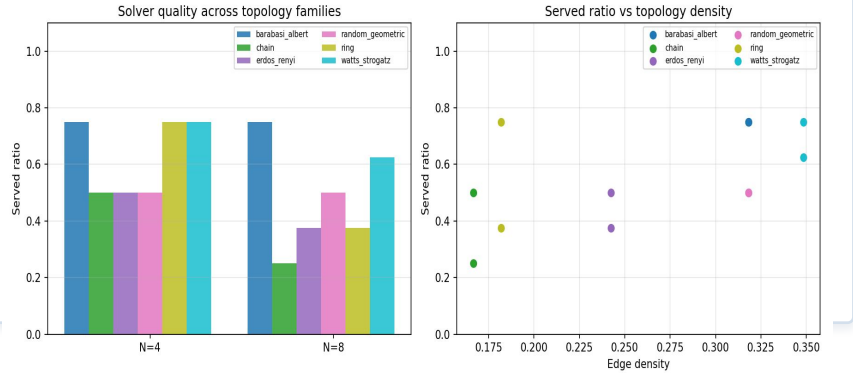
fidelity, generation prob., latency, capacity

Request

source, destination, F□ᵢ□, weight

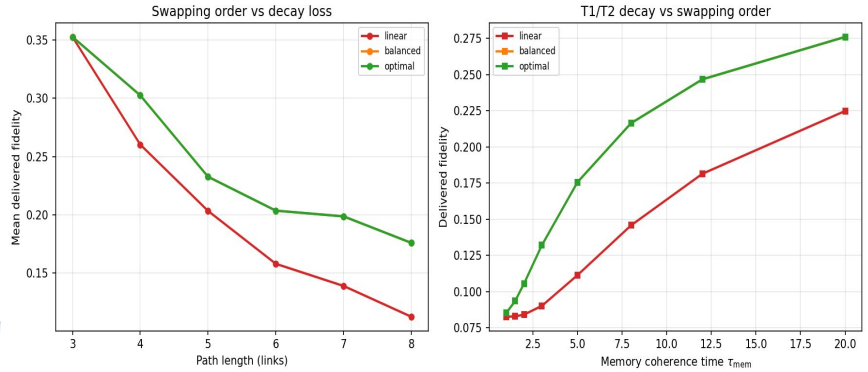
Topology Robustness & Swap-Tree Ordering

Served Ratio Across Topology Families



Solver quality degrades gracefully, only on scale-free (Barabási-Albert) networks.

Delivered Fidelity by Swap-Tree Shape



Balanced/optimal trees recover up to 1.58× delivered fidelity under memory decay.

Served ratio correlates positively with edge density and minimum degree; hub nodes on scale-free graphs become memory/edge bottlenecks.

The QUBO Hamiltonian

$$\mathcal{H} = \mathcal{H}_{\text{util}} + \mathcal{H}_{\text{one}} + \mathcal{H}_{\text{edge}} + \mathcal{H}_{\text{mem}}$$

$$\mathcal{H}_{\text{util}} = -\sum_{r,b} u_{r,b}\, x_{r,b}$$

$$\mathcal{H}_{\text{one}} = A \sum_{r} \left(\sum_{b} x_{r,b} - 1 \right)^2$$

$$\mathcal{H}_{\text{edge}} = B \sum_{e} (L_e + s_e - C_e)^2$$

$$\mathcal{H}_{\text{mem}} = D \sum_{v} (M_v + s_v - M_v^{\max})^2$$

Bundle Utility

$$u_{r,b} = w_r \cdot p_{\text{succ}} \cdot [1 + \beta \max(0,\ F - F_{\min})] - \lambda_\ell \cdot \ell - \lambda_c \cdot \text{cost}$$

- Rewards success probability, weighted by request priority.
- Rewards fidelity margin above the request's F_min requirement.
- Penalizes latency and Bell-pair (resource) expenditure.

Every solver in the stack - Metropolis, tensor-network, OpenJij, embedded quantum annealing minimizes exactly this one Hamiltonian.

x_{r,b} ∈ {0,1}: bundle b selected for request r with A, B, D penalty coefficients and s_e, s_v binary-expanded slack variables

Slack-Free Hamiltonian Formulation

The slack expansion inflates the variable count and couples the penalty scale to the utility range. The direct form replaces it with fixed-scale squared-hinge penalties.

$$\mathcal{H}_{\text{phys}} = -\sum u\,x + \lambda_1 \sum_r \left(\sum_b x_{r,b} - 1 \right)^2$$

$$+ \lambda_2 \sum_e \max(0,\, L_e - C_e)^2 + \lambda_3 \sum_v \max(0,\, M_v - M_v^{\max})^2$$

$\Sigma|B_r|$ binaries

no slack expansion,
exactly one variable per candidate bundle.

Fixed-scale $\lambda_1, \lambda_2, \lambda_3$

penalty multipliers stay anchored to a constant scale,
not the utility range.

$\max(0,\cdot)^2$ penalty

leaves under-capacity configurations unpenalized,
avoiding the low-utility basins slack terms create.

Request-Conditioned Penalty Calibration

Conventional (utility-scale)

$$A_{\text{ref}} = B_{\text{ref}} = D_{\text{ref}} = P_0 + \epsilon$$

All three penalty coefficients are anchored to the largest positive bundle utility P_0, applying the same margin to every resource regardless of how load interacts with capacity.

Every feasible configuration strictly dominates every infeasible one but the same margin applies regardless of load.

Resource-Aware (proposed)

$$\rho^*_{e,b} = \min\{\rho \in \mathcal{R}_{e,-r} : \rho + d_{e,b} > C_e\}$$

$$\delta_{e,b} = [\rho^*_{e,b} + d_{e,b} - C_e]^2_+ - [\rho^*_{e,b} - C_e]^2_+$$

$$B^* = \max_{e,b} \frac{u_b^+}{\delta_{e,b}}, \qquad D^* = \max_{v,b} \frac{u_b^+}{\delta_{v,b}}$$

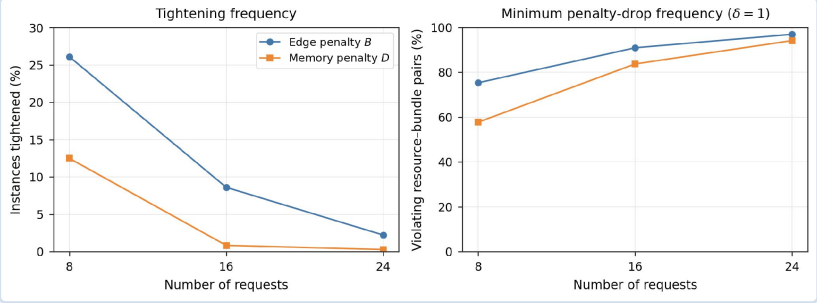
$$A = A_{\text{ref}}, \qquad B = B^* + \epsilon, \qquad D = D^* + \epsilon$$

Provably no larger than the conventional bound, the utility-to-drop ratio can never exceed P_0.

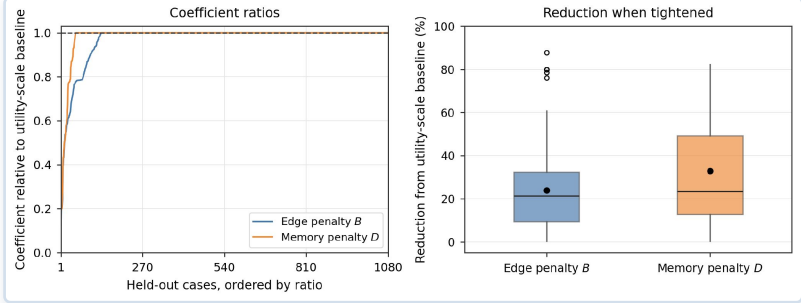
Held-out study (1,080 instances): resource-aware tightens B in 12.3% and D in 4.5% of cases by 23.8% / 32.8% on average where it applies with no detectable change in solver quality.

Penalty Calibration Results

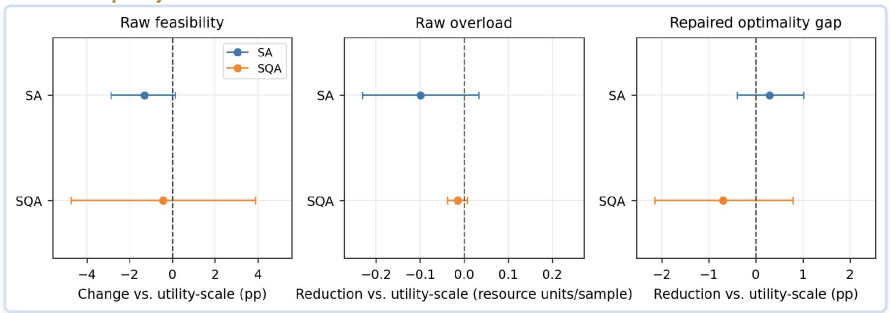
Tightening Frequency & Min. Penalty Drop



Coefficient Ratios vs. Utility-Scale Baseline



Matched OpenJij Effects



Resource-aware calibration provably never loosens sufficient penalties, and tightens B/D by 23.8%/32.8% on average with no systematic solver-quality effect (bootstrap 95% CIs include zero).

Solver Families & Scheduling Architectures

All solvers consume the same Hamiltonian and differ only in search strategy.

M

Metropolis Annealer

Local bit-flip moves, accepted with min(1, e^{-ΔH/T});
geometric cooling T_0=10⟶10^{-3} with a greedy utility-density seed.

⊗

Boundary-MPS Tensor Network

Represents the joint bundle distribution as a matrix product state;
SVD-truncated sweeps contract to the dominant configuration.

Σ

OpenJij Simulated Annealing

Compiles the direct Hamiltonian to a QUBO and samples
with OpenJij's SA sampler;
reference for the adaptive top-k study.

Q

Embedded Quantum Annealing

Minor-embeds the QUBO onto a Chimera-like hardware lattice;
path-integral (PIA) sampler with chain-break-aware decoding.

Stochastic Reliability & Chance-Constrained Routing

Discrete-Event Simulator

Samples the entanglement pipeline event by event:

1. Elementary pair generation (p_gen, retries within capacity)
2. Entanglement swapping (success p_swap)
3. BBPSSW purification
4. Delivery under T1/T2 storage decay
5. Gaussian fidelity noise σ at delivery

REPORTS : Sampled utility $\mathbb{E}[U]$, reliability gap, served ratio, and empirical SLA-violation probability, all from a seeded RNG for deterministic runs.

Chance-Constrained Routing

Replaces the hard rule $F_{r,b} \geq F_{\min}^{(r)}$ with a quantile constraint:

$$\Pr[\, F_r \geq F_{\min} \,] \geq 1 - \varepsilon$$

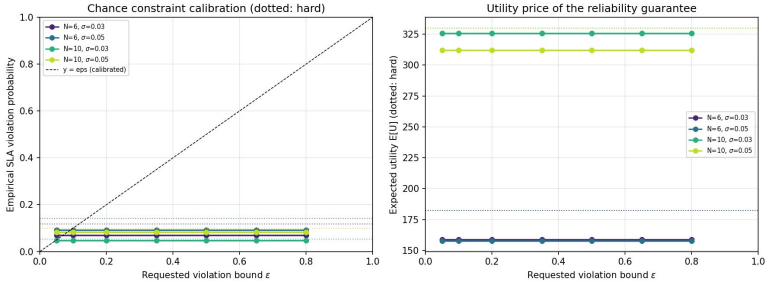
$$\iff \quad F_{r,b} \geq F_{\min} + \Phi^{-1}(1-\varepsilon)\,\sigma$$

A candidate-set filter {b : p_viol(r,b) ≤ ε} followed by the same selection ILP/QUBO, so every solver from the same section applies unchanged.

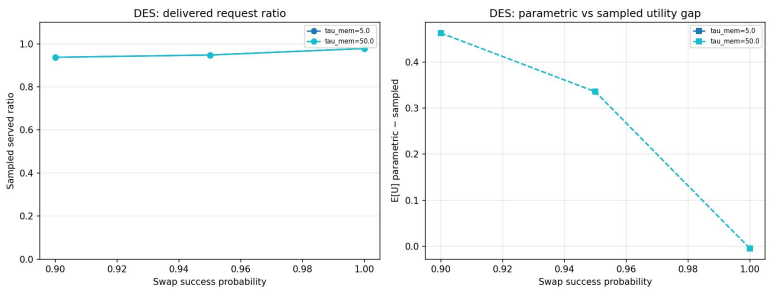
Hard rule implicitly tolerates 5–14% SLA violation;
chance-constrained routing keeps violation ≤ ε on every tested instance.

Stochastic Reliability & Chance-Constrained Routing

Chance-Constrained Routing



DES Reliability Across Coherence Times



5–14%

Implicit SLA-violation rate silently tolerated by the deterministic hard rule

≤ ε

Empirical SLA violation under the chance-constrained rule, on every tested instance

1.2–1.5×

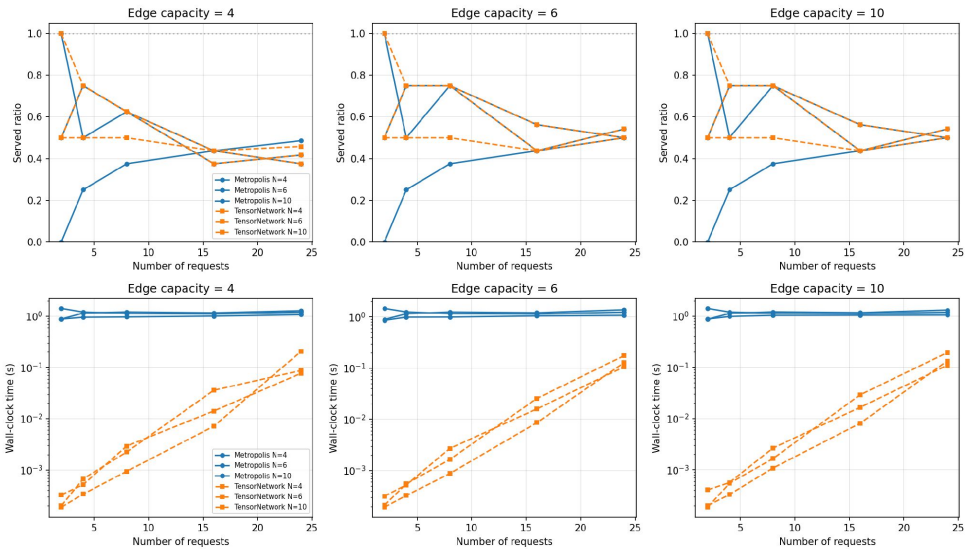
Utility cost of buying the reliability guarantee down to ε = 0.05

0.76

Mean QUBO optimality gap solving the chance-constrained set, no harder than the hard-rule set

Encoding & Core Solver Efficiency

Served Ratio & Wall-Clock Time (Chain Topology)



Metropolis vs. TensorNetwork solvers under varying edge capacity and request count.

4–4,000×

MPS tensor-network solver faster than Metropolis at equal solution quality

1.5%

Mean relative optimality gap of the Metropolis annealer vs. exact ILP

~1%

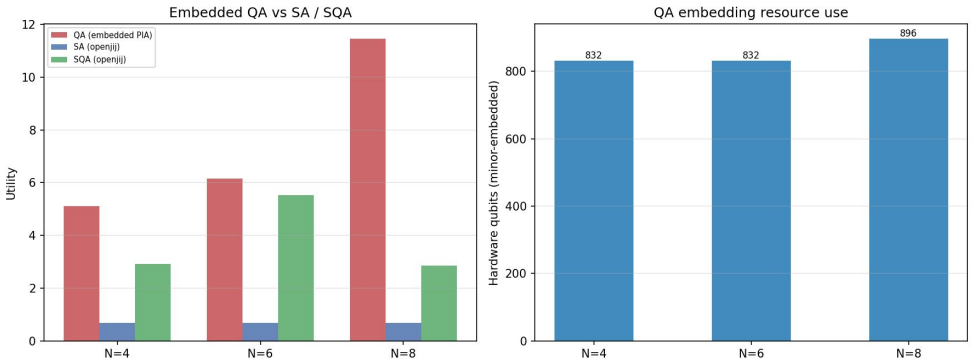
Served-ratio agreement between the MPS and Metropolis solvers

50–150×

Streaming (per-arrival) optimization faster than full re-batching

Hardware-Realistic Quantum Annealing

Embedded QA (PIA) vs. SA and SQA



Minor-embedded onto a Chimera-like hardware lattice; chain-break-aware majority-vote decoding.

832–896

Hardware qubits used by the minor-embedded QUBO, with zero chain breaks

Beats both

The embedded quantum annealer outperforms SA and SQA at every tested size

PIA sampler

Path-integral quantum annealer with chain-break-aware majority-vote decoding

Extension Suite

Nine research extensions plug into the same bundle interface as the core QUBO solvers.

T Temporal-Memory Scheduling

Arrival/deadline/priority requests, T1/T2-aware memory windows

A Adaptive QUBO Reduction

Per-request top-k candidate filtering under a fixed anneal budget

H Hybrid Pipeline

Reduce → solve → repair/refine, chained around the reduced QUBO

G GNN-Guided Ranking

GraphSAGE bundle scorer + feasibility-aware greedy decode

P Pareto / Multi-Objective

Throughput, fidelity, success, latency, memory trade-offs

K k-Disjoint Paths

Composite bundles over edge-disjoint routes with redundancy

S Swap-Tree Ordering

Linear / balanced / optimal swap trees under memory decay

N Generative Topologies

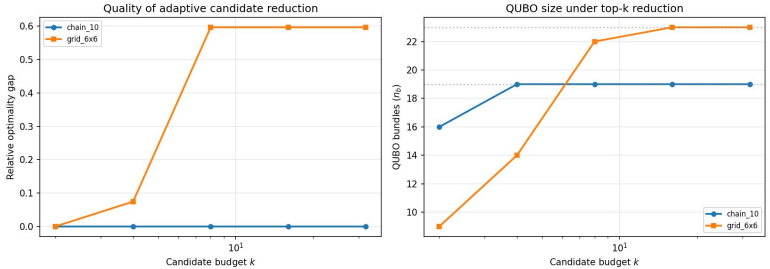
Ring, random-geometric, Erdős–Rényi, Watts–Strogatz, Barabási–Albert

Ψ Purification Scheduling

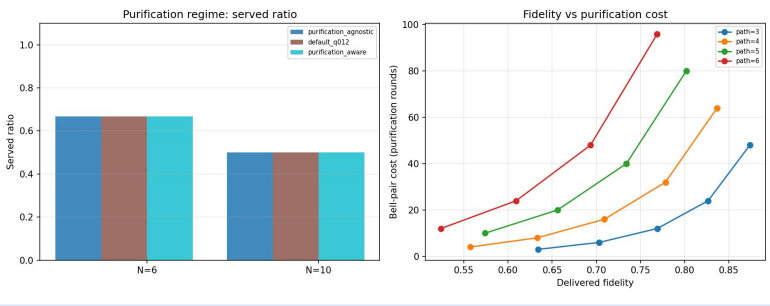
Purification rounds as a first-class optimization variable

Adaptive Reduction, Recourse & Purification

Adaptive Candidate Reduction



Purification as a First-Class Variable



100%

Of failed requests recovered by local repair (recourse) in the DES

334–1,065×

Wall-time speedup of local repair vs. full replanning

0.145 → 0.512

Success probability unlocked by one purification round on a razor-margin bundle

Cost-free

Candidate reduction and purification-aware sets leave the reference optimum unchanged until they bind

CONCLUSION

Key Findings

1. Solver families agree on the capacity-limited served-ratio frontier
   The MPS solver is 4,000× faster. Metropolis is within 1.5% of exact ILP.

2. The direct slack-free Hamiltonian dominates the slack-variable QUBO
   In both size and solution quality.

3. Temporal-memory joint scheduling delivers every admitted request within coherence, at mean fidelity 0.30–0.51.

4. Chance-constrained routing turns the hard rule's implicit 5–14% SLA risk into a validated, tunable ε.

5. Recourse (local repair) recovers 100% of failed requests at 334–1,065× wall-time speedup over full replanning.

6. Candidate reduction, purification, and k-disjoint provisioning are free-or-beneficial until their defined regimes make them bind.

7. Balanced swap trees recover up to 1.58× delivered fidelity under memory decay.

8. Solver quality survives across five topology families, degrading gracefully only on scale-free networks.

All solvers, extensions, experiments, and tests are released as an open-source, fully reproducible suite.

github.com/Sguy1908/QiOpt4QNet-Simulator